Sequential Ordinal Targeting (SOT)

A deterministic method for locating and validating an unencumbered Satoshi locus using UTXO outpoints and ordinal-style offsets

Author: Leon Calvin Long II | Version: 1.0 | Date: 2026/04/02

Proprietary terminology notice. The terms **OMNI-LATTICE Ø** and **Kolmogorov–Shannon Bridge** are proprietary terms defined by the author and used in this document as technical constructs. Their use does not imply affiliation with, endorsement by, or standardization within the Bitcoin protocol or any external standards body.

This document describes an application-layer protocol. Terms such as “Lead Satoshi” and “ordinal” refer to conventions established by ordinal indexers and do not imply changes to Bitcoin’s consensus rules [2,3].

1. Background

Bitcoin represents spendable value as discrete unspent transaction outputs (UTXOs). Each UTXO is uniquely referenced by an *outpoint*, defined as the pair (transaction id, output index). Full nodes maintain a UTXO set—an indexed snapshot of all currently spendable outputs—to validate new transactions efficiently [4].

Ordinal theory is an application-layer numbering and tracking convention that assigns sequential identifiers to Satoshi and describes how satoshis are deterministically assigned from transaction inputs to outputs using a first-in-first-out (FIFO) rule [2]. Indexers and tooling commonly represent an individual Satoshi’s location as a tuple that includes an outpoint plus an offset within that output (e.g., the “first sat” at offset 0) [2,3].

2. Problem Statement

Systems that “anchor” external data to Bitcoin frequently require a deterministic way to choose (a) a specific UTXO under administrative control and (b) a specific Satoshi position within that UTXO, such that the resulting identifier can be reproduced, audited, and proven unused within the system’s own namespace. Without a standard selection and validation procedure, implementations risk ambiguity, collisions, inconsistent indexing across environments, and accidental re-binding of a previously claimed locus.

3. Design Goals

- Determinism: given the same input and state, all implementations select the same target locus.
- Uniqueness: each selected locus maps 1:1 to a system-defined binding record.
- Auditability: third parties can recompute identifiers from public chain data and published outpoints.
- No consensus changes operates entirely at the application layer on top of standard Bitcoin transactions [2].
- Collision resistance within the application: availability is enforced via a local state database.

4. Threat Model and Assumptions

- Bitcoin consensus and transaction validity are out of scope; SOT assumes standard node validation of UTXO spends.
- The system controls (or can reliably enumerate) a set of UTXOs (“treasury wallet”) via wallet software or node RPCs such as *list unspent* [5].
- A local state database tracks previously bound loci and is authoritative for “availability” within the application namespace.
- Adversaries may attempt to induce selection of an already-bound locus, race state updates, or exploit inconsistent indexing; these are mitigated by strict determinism, atomic state updates, and canonical serialization of identifiers.

Abstract

This white paper specifies Sequential Ordinal Targeting (SOT), a deterministic selection protocol for identifying a unique, verifiable “locus” (a specific Satoshi position) inside a Bitcoin unspent transaction output (UTXO). SOT derives a stable identifier from the UTXO outpoint (transaction id and output index) and an offset, then validates availability against a local state database to prevent reuse. The approach enables repeatable selection and auditing without modifying Bitcoin consensus rules and aligns with ordinal-style Satoshi identification conventions used by indexers and wallets. The protocol is intended as an application-layer primitive for binding external data structures to an immutable blockchain reference [1–3].

Keywords: Bitcoin, UTXO, outpoint, ordinals, Satoshi indexing, deterministic selection, data anchoring

5. Protocol Specification

5.1 Terminology

5.1.1 Proprietary Terms

- **OMNI-LATTICE Ø**: A proprietary conceptual model used in this paper to describe the author's internal mapping and allocation structure for loci across a controlled UTXO inventory. Unless explicitly stated otherwise, OMNI-LATTICE Ø is an application-layer construct and not a consensus mechanism.
- **Kolmogorov–Shannon Bridge**: A proprietary conceptual link used in this paper to motivate or interpret the relationship between descriptive complexity and information measures in the context of deterministic locus identification. It is provided as an explanatory framework and does not affect the protocol's normative requirements.
- **UTXO**: An unspent transaction output identified by (txid) and a value in satoshis [4].
- **Outpoint**: The unique reference (txid, out) for a transaction output [4].
- **Offset**: A zero-based index into the satoshis contained in a given output.
- **Lead Satoshi**: The satoshi at offset 0 within an output, consistent with ordinal tooling conventions [2,3].

5.2 Treasury Wallet Model

Let the treasury wallet be modeled as a finite set of spendable UTXOs:

Each UTXO U_i is defined by its outpoint and amount (in satoshis):

$$U_i = (TX_{id}, V_{out}, Amt_{sats})$$

5.3 Locus Identifier Construction

SOT defines a canonical locus identifier by combining the UTXO outpoint with an offset. For the lead Satoshi (offset 0), the locus identifier is:

$$S_{lead} = \text{Concat}(TX_{id}, V_{out}, 0)$$

5.4 Availability Validation

To prevent reuse, the candidate locus is checked against a local state database $\mathcal{D}$ that records all previously bound loci. Implementations should update $\mathcal{D}$ atomically when allocating a new locus to prevent race conditions.

$$V(S_{lead}) = \begin{cases} 1 & \text{if } S_{lead} \notin \mathcal{D} \text{ (Available)} \\ 0 & \text{if } S_{lead} \in \mathcal{D} \text{ (Occupied/Bound)} \end{cases}$$

5.5 Deterministic Target Acquisition

SOT selects the next available locus using a deterministic ordering over the treasury's UTXO inventory. One common policy is FIFO-by-received-order (or another stable sort key such as block height, then txid, then vout), consistent with ordinal tooling's FIFO transfer convention at the satoshi level [2]. The ordering policy MUST be specified and stable across implementations.

$$\tau_{target} = \min\{\mathcal{O}_0(u) \mid u \in \mathbb{W}_{treasury} \wedge \mathcal{O}_0(u) \notin \mathbb{D}_{bound}\}$$

6. Implementation Notes (UTXO Scout)

An implementation typically consists of a "UTXO scout" that enumerates candidate UTXOs from a node or wallet, computes candidate locus identifiers, validates availability, and returns the selected target. Enumeration can be performed using standard wallet/node interfaces (e.g., Bitcoin Core's *listunspent*) [5].

- **Enumerate**: Gather spendable UTXOs (txid, vout, amount, confirmations) from the treasury wallet [5].
- **Construct**: For each candidate, derive the locus identifier (txid ‖ vout ‖ offset). Canonicalize encoding (byte order, delimiters) and document it.
- **Validate**: Check the candidate locus against the local database $\mathcal{D}$ to ensure it is unbound.
- **Select & Reserve**: Choose the first candidate under the stated ordering policy, reserve it by writing to $\mathcal{D}$ atomically, and return it to the caller.

7. Limitations

- Application-layer only: SOT does not create on-chain guarantees about "binding"; it guarantees uniqueness only with respect to the local database $\mathcal{D}$.
- Indexing dependence: if different implementations disagree on canonicalization (e.g., encoding of txid/vout/offset), identifiers may diverge.
- UTXO churn: spending or consolidating treasury UTXOs can invalidate future selection assumptions unless the inventory is refreshed from a node/wallet.
- Privacy considerations: publishing outpoints can reveal wallet structure; systems should consider using dedicated UTXOs for anchoring.

8. Use Cases

- Deterministic anchoring of external records (e.g., document hashes or state commitments) to a reproducible Bitcoin reference.

- Inventory management for systems that allocate unique loci for future inscriptions or metadata conventions.
- Audit trails where an organization wants a stable, public identifier derived from standard chain data (txid/vout) plus a documented offset convention.

9. Conclusion

Sequential Ordinal Targeting (SOT) provides a simple, deterministic procedure to select and validate a unique satoshi locus inside a controlled UTXO set. By grounding identifiers in standard outpoints and a documented offset convention, and by enforcing uniqueness via a local state database, SOT enables reproducible anchoring workflows that can be independently recomputed from public chain data while remaining fully compatible with existing Bitcoin nodes and wallets [2,4,5].

References

1. Hiro Systems, “The Challenges of Building an Indexer for Bitcoin Ordinals,” blog article, 2023.
2. C. Rodarmor, *Ordinal Theory Handbook*, documentation site, 2023 (and updates).
3. ordinals/ord contributors, “Sat Hunting,” *ord* documentation, 2023.
4. Blockstream, “UTXO Set,” glossary entry, accessed 2026.
5. Bitcoin Developer Documentation, “listunspent” RPC reference, accessed 2026.